

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA0601	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)		
Student Name:	V. Prabhavathi	Reg. No.:	192472067
Section / Batch:	CSE-A1	Date:	20/07/26

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

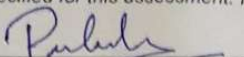
Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:			100 Marks

SECTION A – Technical Presentation

(Handwritten signature/initials)

Code of Conduct

I, V. Prabhavathi (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

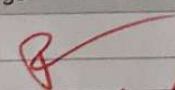
Signature of the Student: 

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1	30	25	20	14	9	48
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: <u></u>	Signature: _____	Signature: _____
Date: <u>20/7/26</u>	Date: _____	Date: _____

Col- Assessment Tool- 3
Technical Presentation

U. Prabhavathi
192472067
CSA0601

97

Case Analysis of Binary Search Efficiency

Problem Definition

Binary search is used to search an element in a sorted array by repeatedly dividing the search space into two halves

Best case

- * Element is found in the first comparison
- * Comparisons = 1
- * Complexity: $O(1)$

Average case

- * Element is found after several divisions
- * Comparison $\approx \log_2 n$
- * Complexity: $O(\log n)$

Worst case

- * Element is at the last possible position or not Present.
- * Comparison = $\log_2 n$
- * Complexity: $O(\log n)$

Complexity Table

Case	Time Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

Interpretation

Binary search is highly efficient because it reduces the search space by half after every comparison, making it much faster than linear search

97 Space complexity of Breadth-First Traversal

Problem Definition

Breadth-first search (BFS) visits nodes level by level using a queue.

Memory Usage

- * Queue stores nodes waiting to be processed
- * Maximum memory occurs at the widest level of the tree

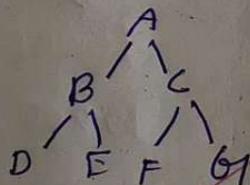
Space Complexity

- worst case: $O(n)$
- Best case: $O(1)$ (very small tree)

Table

Component	Space
Queue	$O(n)$
Variables	$O(1)$
Total	$O(n)$

Example Tree



BFS Traversal order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Interpretation

BFS requires more memory than DFS because many nodes may be stored in the queue at the same time.

98 Efficiency Analysis of Pairwise Comparison Algorithm

Problem Definition

Each element is compared with every other element.

No. of comparisons:-

$$\frac{n(n-1)}{2}$$

Time complexity

Case complexity

Best $O(n^2)$

Average $O(n^2)$

Worst $O(n^2)$

Ex:- For $n=5$

$$\text{Comparisons} = 5 \times 4 / 2 = 10$$

Space Complexity

The algorithm uses only a few temporary Variables.

$$\text{Space complexity} = O(1)$$

Applications

- * Duplicate detection
- * Similarity matching
- * Ranking and Voting systems

Interpretation

The number of comparisons grows rapidly as input size increases, making pairwise comparison

29 Divide and conquer efficiency comparison

Definition

- * Dividing the problem into smaller subproblems
- * Solving each subproblem recursively
- * Combining the solutions to obtain the final result

Problem statement

Compare the following recurrence relations using the Master Theorem:

1. $T(n) = 2T(n/2) + n$

2. $T(n) = 2T(n/2) + n \log n$

Recurrence 1

$$T(n) = 2T(n/2) + n$$

Identify Parameters

$$a=2, b=2, f(n)=n$$

Calculate

$$n \log_b a = n \log_2 2 = n$$

$$f(n) = n = n \log_b a$$

Final complexity

$$\boxed{O(n \log n)}$$

Recurrence 2

$$T(n) = 2T(n/2) + n \log n$$

$$a=2, b=2, f(n)=n \log n$$

$$f(n) = \Theta(n \log_b a \log n)$$

$$\boxed{O(n \log^2 n)}$$

Comparison Table

Recurrence Relation	Combine Cost	Time complexity
$T(n) = 2T(n/2) + n$	$O(n)$	$O(n \log n)$
$T(n) = 2T(n/2) + n \log n$	$O(n \log n)$	$O(n \log^2 n)$

Interpretation:-

The first recurrence is more efficient because its combine step costs only $O(n)$ instead of $O(n \log n)$.

20 Visualization - Based study of growth Functions

Common growth Functions

Function	Complexity
constant	$O(1)$
logarithmic	$O(\log n)$
Linear	$O(n)$
Linearithmic	$O(n \log n)$
Quadratic	$O(n^2)$

Problem statement

- $O(1)$ - constant time
- $O(\log n)$ - logarithmic Time
- $O(n)$ - linear time
- $O(n \log n)$ - linearithmic Time
- $O(n^2)$ - Quadratic time

Comparison Table

Input(n)	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
10	1	3	10	33	10
100	1	7	100	664	10000
1000	1	10	1000	9966	1,000,000

Applications

- * choosing efficient Sorting algorithm
- * Performance analysis of searching techniques

Interpretation:

- $O(1)$: Fastest and does not depend on input size
- $O(\log n)$: Very efficient
- $O(n)$: Acceptable for most applications.